DOM con JavaScript

Esta guía explica, paso a paso y con ejemplos, cómo usar JavaScript para leer y cambiar una página web mientras el usuario la usa.

1. ¿Qué es el DOM?

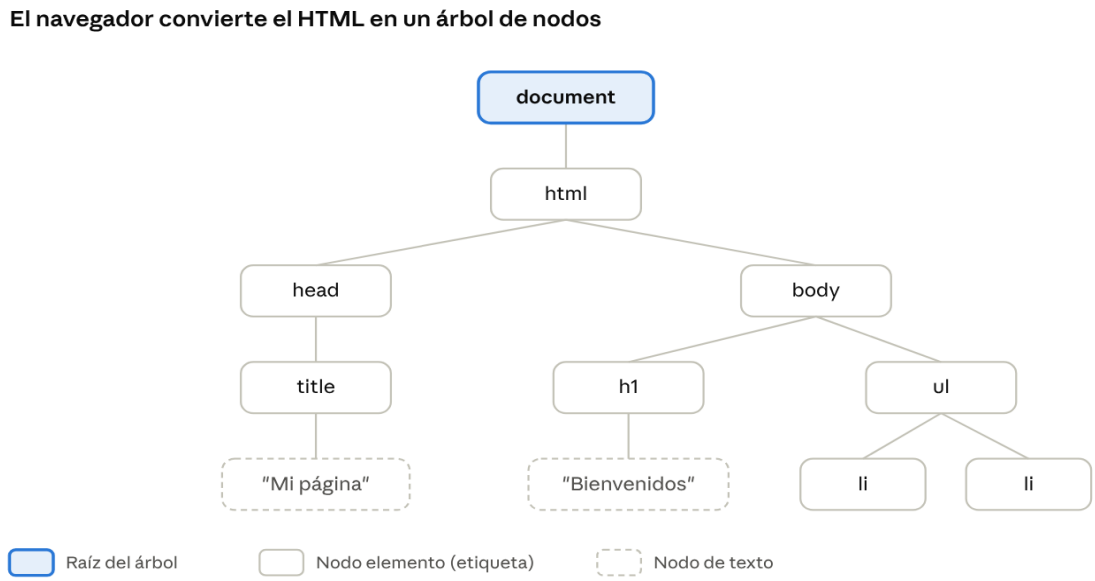
El DOM (Document Object Model) es la copia en memoria que el navegador hace de tu HTML, organizada como un árbol que JavaScript puede leer y modificar.

Cuando el navegador abre una página, no trabaja directamente con el texto del archivo `.html`. Lo convierte en objetos: cada etiqueta se vuelve un **nodo elemento** y cada texto dentro de ella, un **nodo de texto**. Si JavaScript cambia uno de esos objetos, la pantalla se actualiza al instante, sin recargar.

Por ejemplo, este HTML:

```
<html>
 <head>
  <title>Mi página</title>
 </head>
 <body>
  <h1>Bienvenidos</h1>
  <ul>
   <li>Uno</li>
   <li>Dos</li>
  </ul>
 </body>
</html>
```

se convierte en este árbol:



árbol DOM del HTML de ejemplo

Las relaciones se nombran como en una familia: `body` es **padre** de `h1` y `ul`, ellos son sus **hijos**, y los dos `li` son **hermanos**. Toda búsqueda en el DOM empieza desde el objeto `document`, la raíz.

2. Preparar el entorno

Solo necesitas un editor (Visual Studio Code), un navegador (Chrome o Firefox) y la extensión Live Server para ver los cambios al guardar.

Organiza tu proyecto así:

```
practica-dom/
├── index.html
├── css/estilos.css
└── js/app.js
```

Enlaza el script con `defer`. Así el navegador termina de construir el DOM antes de ejecutar tu código; sin él, JavaScript buscaría elementos que todavía no existen y obtendría `null`.

```
<head>
  <link rel="stylesheet" href="css/estilos.css">
  <script src="js/app.js" defer></script>
</head>
```

Tu mejor herramienta: las DevTools. Presiona F12 en cualquier página:

- En **Elements** ves el árbol DOM en vivo y puedes editarlo para probar.
- En **Console** puedes escribir JavaScript directamente. Prueba `document.title` o `document.body.children`.
- Usa `console.log()` en tu código para revisar qué valor tiene cada variable.

3. Seleccionar elementos

Antes de cambiar algo, hay que encontrarlo. Hoy basta con dos métodos: `querySelector` para un elemento y `querySelectorAll` para varios; ambos usan los mismos selectores que CSS.

Método	Devuelve	Ejemplo
`querySelector()`	El primer elemento que cumple el selector, o `null`	`document.querySelector('.tarjeta')`
`querySelectorAll()`	Una NodeList con todos los que lo cumplen	`document.querySelectorAll('li')`
`getElementById()`	El elemento con ese id, o `null`	`document.getElementById('titulo')`
`getElementsByClassName()`	Una colección "viva" que se actualiza sola	`document.getElementsByClassName('item')`

```
const titulo = document.querySelector('#titulo');   // por id
const primera = document.querySelector('.tarjeta');  // por clase
const enlaces = document.querySelectorAll('nav a');  // varios

enlaces.forEach((enlace) => {
  console.log(enlace.textContent);
});
```

También puedes moverte por el árbol desde un elemento que ya tienes:

```
const item = document.querySelector('li');
item.parentElement;          // el ul que lo contiene
item.nextElementSibling;     // el siguiente li
item.closest('.menu');       // el ancestro más cercano con esa clase
```

Pruébalo: en una página con una lista de cinco frutas, muestra en consola el nombre de cada una recorriendo la NodeList con `forEach`.

4. Modificar contenido, atributos y estilos

Una vez tienes el elemento, cambiarlo es asignar una propiedad.

Texto y HTML interno

```
const parrafo = document.querySelector('p');
parrafo.textContent = 'Texto nuevo';         // solo texto
parrafo.innerHTML = 'Texto <strong>nuevo</strong>'; // interpreta HTML
```

Usa `textContent` siempre que el contenido venga del usuario. Con `innerHTML`, alguien podría escribir código malicioso en un campo y el navegador lo ejecutaría (ataque XSS).

Atributos

```
const imagen = document.querySelector('img');
imagen.setAttribute('alt', 'Foto de perfil');
imagen.src = 'img/perfil.png';
imagen.getAttribute('alt');   // 'Foto de perfil'
imagen.removeAttribute('title');
```

Clases y estilos. Lo recomendado es definir el aspecto en CSS y desde JavaScript solo poner o quitar clases:

```
const caja = document.querySelector('.caja');
caja.classList.add('activa');
caja.classList.remove('activa');
caja.classList.toggle('oculta');     // pone si no está, quita si está
caja.classList.contains('activa');    // true o false

caja.style.backgroundColor = '#39A900'; // estilo directo, para casos puntuales
```

Pruébalo: crea un botón "Modo oscuro" que alterne la clase `oscuro` en el `body`, con sus colores definidos en CSS.

5. Crear, insertar y eliminar elementos

Crear un elemento tiene tres pasos: crearlo, darle contenido e insertarlo en la página. Hasta el tercer paso no se ve.

```
const lista = document.querySelector('#tareas');

// 1. Crear
const item = document.createElement('li');
// 2. Darle contenido
item.textContent = 'Estudiar el DOM';
item.classList.add('tarea');
// 3. Insertar
lista.append(item);
```

Método	Dónde inserta
padre.append(el)	Al final, dentro del padre
padre.prepend(el)	Al inicio, dentro del padre
el.before(nuevo)	Justo antes de el, como hermano
el.after(nuevo)	Justo después de el, como hermano
el.replaceWith(nuevo)	En lugar de el
el.remove()	Elimina el de la página

Generar elementos desde datos. Es el caso más común: tienes un arreglo y quieres una tarjeta por cada objeto.

```
const productos = [
  { nombre: 'Teclado', precio: 80000 },
  { nombre: 'Mouse', precio: 45000 },
];
const catalogo = document.querySelector('#catalogo');

productos.forEach((p) => {
  const tarjeta = document.createElement('article');
  tarjeta.classList.add('tarjeta');

  const nombre = document.createElement('h3');
  nombre.textContent = p.nombre;

  const precio = document.createElement('p');
  precio.textContent = `$${p.precio.toLocaleString('es-CO')}`;

  tarjeta.append(nombre, precio);
```

```
  catalogo.append(tarjeta);
});
```

Para vaciar un contenedor antes de volver a dibujarlo: `catalogo.replaceChildren();`

6. Eventos y delegación de eventos

Un evento es algo que pasa en la página (un clic, una tecla, un envío) y `addEventListener` le dice al navegador qué función ejecutar cuando ocurra.

```
const boton = document.querySelector('#contar');
let clics = 0;

boton.addEventListener('click', () => {
  clics++;
  boton.textContent = `Clics: ${clics}`;
});
```

Evento	Cuándo ocurre
`click`	Al hacer clic en el elemento
`input`	Cada vez que cambia el texto de un campo
`change`	Al cambiar un `select`, un checkbox o al salir de un campo
`submit`	Al enviar un formulario
`keydown`	Al presionar una tecla
`mouseover` / `mouseout`	Al entrar o salir el puntero del elemento
`DOMContentLoaded`	Cuando el árbol DOM ya está listo

El objeto evento. La función recibe un parámetro (por costumbre `e`) con información de lo que pasó: `e.target` es el elemento donde ocurrió y `e.key` la tecla presionada.

```
document.addEventListener('keydown', (e) => {
  if (e.key === 'Escape') cerrarModal();
});
```

Delegación de eventos. Si tienes una lista con muchos botones, o botones que se crean después, no le pongas un listener a cada uno. Ponle uno solo al contenedor y pregunta dónde fue el clic:

```
const lista = document.querySelector('#tareas');

lista.addEventListener('click', (e) => {
  if (e.target.matches('.btn-eliminar')) {
    e.target.closest('li').remove();
  }
});
```

Esto funciona porque los eventos "suben" (burbujean) desde el elemento donde ocurrieron hasta sus ancestros.

Pruébalo: un contador con botones "+", "−" y "Reiniciar" que nunca baje de cero.

7. Formularios y validación

Para manejar un formulario con JavaScript escucha el evento `submit` y llama a `e.preventDefault()`, que evita que la página se recargue.

```
<form id="registro">
  <label for="correo">Correo</label>
  <input id="correo" name="correo" type="email">
  <small id="error"></small>
  <button type="submit">Enviar</button>
</form>
```

```
const form = document.querySelector('#registro');
const error = document.querySelector('#error');

form.addEventListener('submit', (e) => {
  e.preventDefault();
  const correo = form.correo.value.trim();

  if (!correo.includes('@')) {
    error.textContent = 'Ingresa un correo válido';
    form.correo.focus();
    return;
  }

  error.textContent = '';
  console.log('Enviado:', correo);
  form.reset();
});
```

Puntos clave:

- `.value` lee lo que escribió el usuario; `.trim()` quita espacios sobrantes.
- Muestra el error junto al campo, no con `alert()`.
- Para validar mientras se escribe, usa el evento `input` en el campo.
- La validación en el navegador ayuda al usuario, pero el servidor siempre debe validar de nuevo.

8. Guardar datos con localStorage

`localStorage` guarda datos en el navegador para que sigan ahí al recargar la página. Solo almacena texto, así que los arreglos y objetos se convierten con JSON.

```
// Guardar
const tareas = ['Estudiar', 'Practicar'];
localStorage.setItem('tareas', JSON.stringify(tareas));
```

```
// Leer (con valor por defecto si no hay nada guardado)
const guardadas = JSON.parse(localStorage.getItem('tareas')) ?? [];

// Borrar
localStorage.removeItem('tareas');
```

Los datos quedan solo en ese navegador y ese equipo. No guardes ahí contraseñas ni información sensible.

9. Ejemplo completo: gestor de tareas

Este ejemplo une todo lo anterior: agrega tareas desde un formulario, las marca como hechas, las elimina, las filtra, cuenta las pendientes y las guarda en `localStorage`.

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Gestor de tareas</title>
 <link rel="stylesheet" href="css/estilos.css">
 <script src="js/app.js" defer></script>
</head>
<body>
 <h1>Mis tareas</h1>

 <form id="form-tarea">
  <label for="nueva">Nueva tarea</label>
  <input id="nueva" type="text" placeholder="Escribe una tarea">
  <button type="submit">Agregar</button>
 </form>

 <div id="filtros">
  <button data-filtro="todas" class="activo">Todas</button>
  <button data-filtro="pendientes">Pendientes</button>
  <button data-filtro="hechas">Hechas</button>
 </div>

 <ul id="lista"></ul>
 <p id="contador"></p>
</body>
</html>
```

css/estilos.css (lo mínimo)

```
.hecha span { text-decoration: line-through; opacity: 0.6; }
#filtros .activo { font-weight: bold; }
```

Script.js

```
// 1. Referencias a los elementos
const form = document.querySelector('#form-tarea');
const input = document.querySelector('#nueva');
const lista = document.querySelector('#lista');
const contador = document.querySelector('#contador');
const filtros = document.querySelector('#filtros');

// 2. Estado: los datos son la fuente de verdad
let tareas = JSON.parse(localStorage.getItem('tareas')) ?? [];
let filtro = 'todas';

function guardar() {
  localStorage.setItem('tareas', JSON.stringify(tareas));
}

// 3. Dibujar la lista a partir de los datos
function render() {
  lista.replaceChildren();

  const visibles = tareas.filter((t) => {
    if (filtro === 'pendientes') return !t.hecha;
    if (filtro === 'hechas') return t.hecha;
    return true;
  });

  visibles.forEach((t) => {
    const li = document.createElement('li');
    li.dataset.id = t.id;
    li.classList.toggle('hecha', t.hecha);

    const check = document.createElement('input');
    check.type = 'checkbox';
    check.checked = t.hecha;
    check.classList.add('check');
    check.setAttribute('aria-label', 'Marcar como hecha');

    const texto = document.createElement('span');
    texto.textContent = t.texto;

    const borrar = document.createElement('button');
    borrar.textContent = 'Eliminar';
    borrar.classList.add('btn-eliminar');
```

```
    li.append(check, texto, borrar);
    lista.append(li);
  });

  const pendientes = tareas.filter((t) => !t.hecha).length;
  contador.textContent = `${pendientes} pendiente(s)`;
}

// 4. Agregar
form.addEventListener('submit', (e) => {
  e.preventDefault();
  const texto = input.value.trim();
  if (!texto) return;

  tareas.push({ id: Date.now(), texto, hecha: false });
  input.value = '';
  guardar();
  render();
});

// 5. Marcar y eliminar (delegación de eventos)
lista.addEventListener('click', (e) => {
  const li = e.target.closest('li');
  if (!li) return;
  const id = Number(li.dataset.id);

  if (e.target.matches('.check')) {
    const tarea = tareas.find((t) => t.id === id);
    tarea.hecha = e.target.checked;
  } else if (e.target.matches('.btn-eliminar')) {
    tareas = tareas.filter((t) => t.id !== id);
  } else {
    return;
  }
  guardar();
  render();
});

// 6. Filtrar
filtros.addEventListener('click', (e) => {
  if (!e.target.matches('button')) return;
  filtro = e.target.dataset.filtro;
  filtros.querySelectorAll('button').forEach((b) => {
    b.classList.toggle('activo', b === e.target);
  });
  render();
});
```

```
render();
```

La idea central: el arreglo `tareas` es la fuente de verdad. Cada acción cambia los datos, los guarda y llama a `render()`, que vuelve a dibujar la lista. Así la pantalla nunca queda desincronizada de los datos; es el mismo principio que usan React o Vue.

Retos para practicar:

- Editar el texto de una tarea con doble clic.
- Botón "Borrar completadas".
- Mostrar un mensaje cuando la lista esté vacía.
- Agregar fecha límite y resaltar las vencidas.

10. Errores comunes y buenas prácticas

Error o síntoma	Causa	Solución
`Cannot read properties of null`	El script corre antes de que exista el elemento, o el selector está mal escrito	Usa `defer` y revisa el selector en la consola
`forEach is not a function`	Usaste `getElementsByClassName`, que no siempre tiene `forEach`	Usa `querySelectorAll` o `Array.from()`
El listener no funciona en elementos nuevos	Se asignó antes de crearlos	Usa delegación de eventos en el contenedor
La página se recarga al enviar el formulario	Falta `e.preventDefault()`	Agrégalo al inicio del manejador de `submit`
`localStorage` devuelve `[object Object]`	Se guardó un objeto sin convertir	Usa `JSON.stringify` al guardar y `JSON.parse` al leer
Texto del usuario rompe la página	Se insertó con `innerHTML`	Usa `textContent`

Buenas prácticas:

- Guarda en constantes los elementos que usas varias veces, en lugar de buscarlos en cada evento.
- Cambia clases desde JavaScript y deja el diseño en CSS.
- Mantén los datos en variables y dibuja la interfaz a partir de ellos.
- Usa nombres descriptivos: `btnAgregar` en lugar de `b1`.
- Usa `label` en los campos y botones reales (`button`) para que la página funcione con teclado y lector de pantalla.

11. Para seguir aprendiendo

- MDN: Introducción al DOM
- MDN: Introducción a los eventos
- MDN: Web Storage API
- El Tutorial de JavaScript Moderno: Documento
- Eloquent JavaScript, capítulos 14 y 15